

PRAKTIKUM PERTEMUAN 11

STRUKTUR DATA & ALGORITMA

Nama: Lumaris Satya Dwinanto

Nim: J0403251143

Kelas: TPL A

Praktikum 1 - Adjacency Matrix

Output:

```
PS C:\Users\ARIS\OneDrive\Dokumen\Kulyeah\Struktur dan logaritma\Pertemuan 1\ASD_J0403251143_Lumaris Satya Dwinanto\Praktikum 11> & C:/Users/ARIS/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/ARIS/OneDrive/Dokumen/Kulyeah/Struktur dan logaritma/Pertemuan 1/ASD_J0403251143_Lumaris Satya Dwinanto/Praktikum 11/Praktikum1 - Membuat Adjacency Matrix.py"
Adjacency Matrix Representation:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
PS C:\Users\ARIS\OneDrive\Dokumen\Kulyeah\Struktur dan logaritma\Pertemuan 1\ASD_J0403251143_Lumaris Satya Dwinanto\Praktikum 11>
```

Praktikum 2 - Adjacency List

Output:

```
PS C:\Users\ARIS\OneDrive\Dokumen\Kulyeah\Struktur dan logaritma\Pertemuan 1\ASD_J0403251143_Lumaris Satya Dwinanto\Praktikum 11> & C:/Users/ARIS/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/ARIS/OneDrive/Dokumen/Kulyeah/Struktur dan logaritma/Pertemuan 1/ASD_J0403251143_Lumaris Satya Dwinanto/Praktikum 11/Praktikum 2- Membuat Adjacency List.py"
Adjacency List Representation:
A: B C
B: A D
C: A D
D: B C
PS C:\Users\ARIS\OneDrive\Dokumen\Kulyeah\Struktur dan logaritma\Pertemuan 1\ASD_J0403251143_Lumaris Satya Dwinanto\Praktikum 11>
```

Praktikum 3: Adjacency Matrix

Output:

```
PS C:\Users\ARIS\OneDrive\Dokumen\Kulyeah\Struktur dan logaritma\Pertemuan 1\ASD_J0403251143_Lumaris Satya Dwinanto\Praktikum 11> & C:/Users/ARIS/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/ARIS/OneDrive/Dokumen/Kulyeah/Struktur dan logaritma/Pertemuan 1/ASD_J0403251143_Lumaris Satya Dwinanto/Praktikum 11/Praktikum 3 - Konversi Matrix ke List.py"
Hasil Konversi Adjacency Matrix ke Adjacency List:
0: [1, 2]
1: [0, 2]
2: [0, 1, 3]
3: [2]
PS C:\Users\ARIS\OneDrive\Dokumen\Kulyeah\Struktur dan logaritma\Pertemuan 1\ASD_J0403251143_Lumaris Satya Dwinanto\Praktikum 11>
```

Praktikum 4 – Studi Kasus Dunia Nyata

Output:

```
--- OUTPUT PROGRAM STUDI KASUS JARINGAN KOMPUTER ---
Nama Node (Vertex): ['Router_A', 'Router_B', 'Switch_1', 'Server_1', 'PC_1']
Hubungan Antar Node (Edge): [('Router_A', 'Router_B'), ('Router_A', 'Switch_1'), ('Router_B', 'Switch_1'), ('Switch_1', 'Server_1'), ('Switch_1', 'PC_1'), ('Router_B', 'Server_1')]
-----
1. Adjacency List:
Router_A: Router_B Switch_1
Router_B: Router_A Switch_1 Server_1
Switch_1: Router_A Router_B Server_1 PC_1
Server_1: Switch_1 Router_B
PC_1: Switch_1

2. Adjacency Matrix:
0 1 1 0 0
1 0 1 1 0
1 1 0 1 1
0 1 1 0 0
0 0 1 0 0
PS C:\Users\ARIS\OneDrive\Dokumen\Kulyeah\Struktur dan logaritma\Pertemuan 1\ASD_30403251143_Lumaris Satya Dwinanto\Praktikum 11>
```

1. Penentuan Node dan Edge

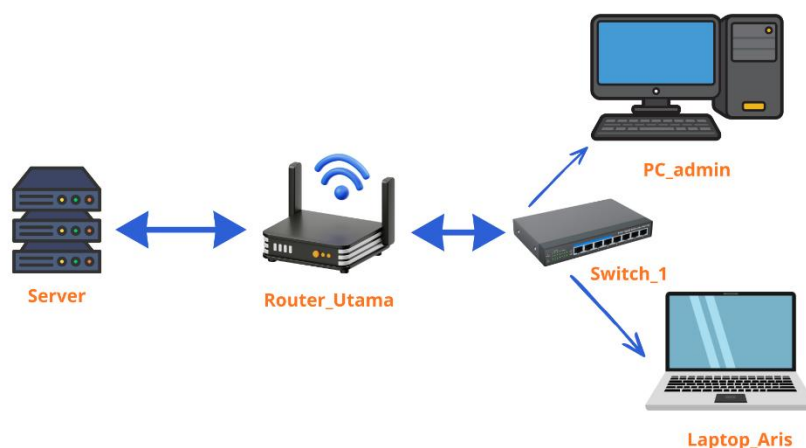
Node (Vertex): Perangkat keras dalam jaringan.

Saya pakai 5 node: Router_Utama, Server, PC_Admin, Laptop_Aris, dan Switch_1.

2. Gambar Desain Graph:

Saya buat topologi di mana semua terpusat ke Router dan Switch:

1. **Router_Utama** terhubung ke **Server** dan **Switch_1**.
2. **Switch_1** terhubung ke **PC_Admin**, **Laptop_Aris**, dan **Router_Utama**.
3. **Server** terhubung ke **Router_Utama**.



Analisis Singkat:

1. Penjelasan Node dan Edge: Pada studi kasus Jaringan Komputer ini, *Node* merepresentasikan perangkat keras jaringan yaitu Router_A, Router_B, Switch_1, Server_1, dan PC_1. Sedangkan *Edge* merepresentasikan kabel fisik (misalnya kabel UTP/Fiber) yang menghubungkan perangkat-perangkat tersebut untuk saling berkomunikasi. Total terdapat 5 node dan 6 edge sesuai dengan syarat minimal.

2. Jenis Graph: Graph ini termasuk jenis Undirected Graph (tidak berarah). Alasannya karena komunikasi data melalui kabel LAN pada topologi ini bersifat dua arah (bidirectional); jika Router_A dapat mengirim paket data ke Switch_1, maka Switch_1 juga dapat mengirim kembali ke Router_A.

3. Kepadatan Graph (Dense atau Sparse): Graph ini tergolong **Sparse Graph** (renggang). Hal ini karena jumlah koneksi/edge yang ada (6 koneksi) relatif lebih sedikit dibandingkan dengan kemungkinan total koneksi maksimal jika semua perangkat saling terhubung langsung satu sama lain secara penuh (*full mesh*). Router_A.

4. Representasi yang Lebih Cocok: Berdasarkan sifat graph yang *sparse*, representasi yang paling cocok digunakan adalah **Adjacency List**. Adjacency List jauh lebih hemat penggunaan memori dengan kompleksitas ruang $O(V+E)$ dibandingkan Adjacency Matrix $O(V^2)$. Jika menggunakan matriks, akan ada terlalu banyak sel bernilai 0 yang terbuang sia-sia karena tidak semua perangkat jaringan terkoneksi secara langsung satu sama lain.

Kesimpulan:

Melalui praktikum ini, dapat disimpulkan bahwa Graph merupakan struktur data non-linear yang memodelkan hubungan antar objek pada kasus nyata, seperti perangkat pada topologi jaringan komputer. Terdapat dua representasi utama yang dipelajari menggunakan Python, yaitu Adjacency Matrix dan Adjacency List.

Adjacency Matrix, yang direpresentasikan menggunakan matriks 2 dimensi, unggul dalam kecepatan pengecekan koneksi $O(1)$ sehingga ideal untuk *dense graph*. Namun, metode ini membutuhkan ruang memori yang besar $O(V^2)$. Sebaliknya, Adjacency List lebih hemat memori dengan kompleksitas ruang $O(V+E)$ dan sangat cocok diterapkan pada *sparse graph*, seperti jaringan komputer pada studi kasus ini di mana tidak semua perangkat terhubung langsung satu sama lain. Pemilihan representasi ini sangat penting untuk memastikan efisiensi algoritma berdasarkan kepadatan sisi (edge) pada sistem yang dirancang.